HPC Assignment 04: Particle Interpolation and Mover Performance Analysis

Vansh Patel
Roll Number: 202301443

Manya Shah
Roll Number: 202301424

February 25, 2026

Abstract

This report builds upon the scattered point to mesh interpolation scheme from Assignment 03. We evaluate its performance by exploring the scaling of execution time with varying particle counts, analyzing interpolation time consistency across iterations, and introducing a particle Mover phase. The Mover operation is subsequently parallelized using OpenMP, and performance is compared between a local Lab PC and an HPC cluster.

1 Introduction and Implementation Approach

The simulation domain consists of particles and grid points strictly bounded within a 1×1 area. The simulation proceeds in two main phases:

1. **Interpolation:** Distributing particle values to the mesh grid using bilinear area-based weighting.
2. **Mover Operation:** Displacing particles by random amounts within $\pm\Delta x$ and $\pm\Delta y$, where $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. If a particle's new position falls outside the domain boundary ($0.0 \leq x, y \leq 1.0$), the displacement is regenerated until the position is valid.

The Mover phase is parallelized using standard OpenMP directives with a fixed thread count of 4.

2 Overall Approach (Pictorial)

The schematic above outlines the entire algorithmic flow, from initialization through the serial interpolation phase and into the OpenMP parallelized mover phase.

3 Experiment 01: Scaling with Number of Particles

This experiment observes how interpolation execution time scales with particle counts ranging from 10^2 to 10^9 for $Maxiter = 10$. Timings strictly exclude memory allocation.

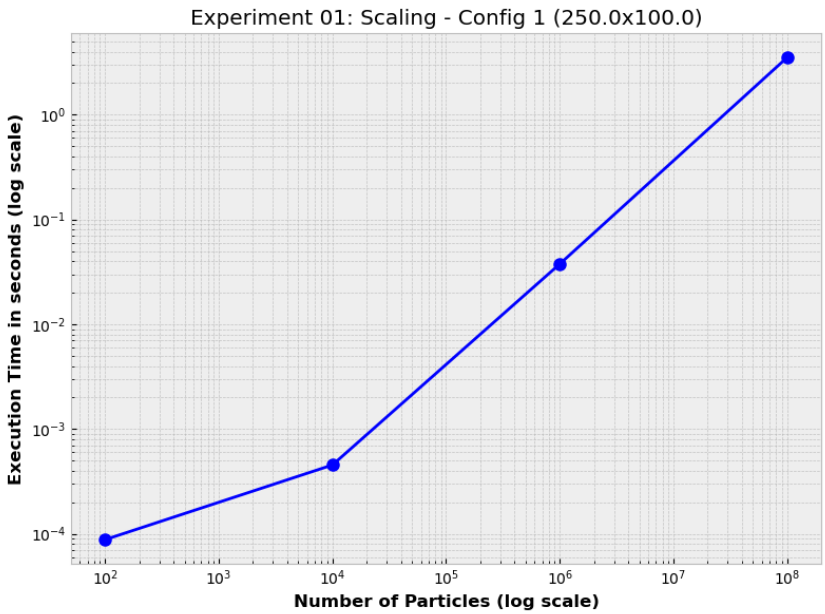

Figure 1: Scaling for Configuration 1 ($N_x = 250$, $N_y = 100$)

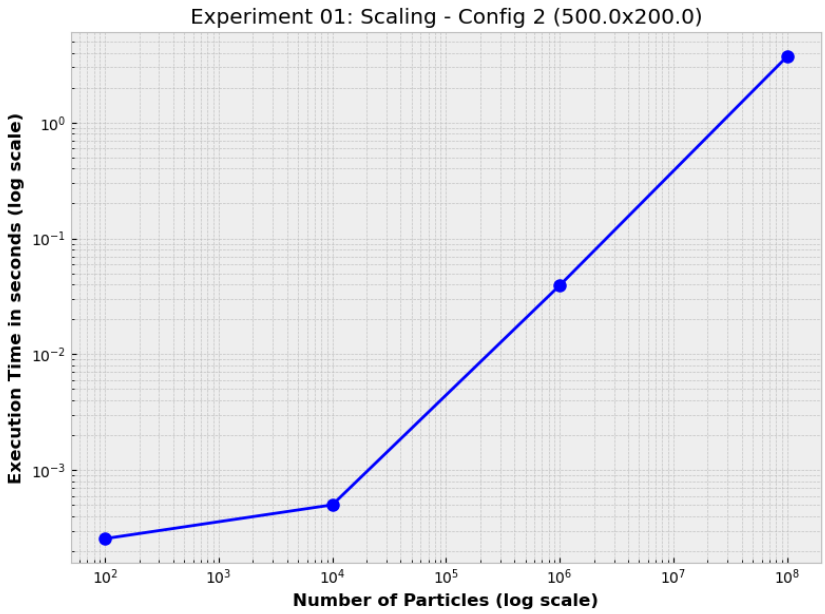

Figure 2: Scaling for Configuration 2 ($N_x = 500$, $N_y = 200$)

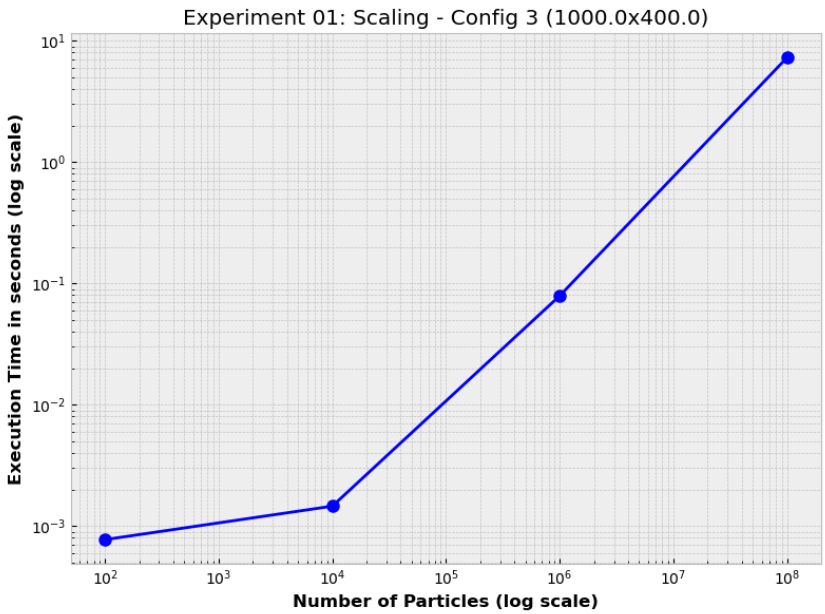

Figure 3: Scaling for Configuration 3 ($N_x = 1000$, $N_y = 400$)

4 Experiment 02: Consistency Across Configurations

To test consistency, particles were fixed to 10^8 and initialized only once before entering the main loop for 10 iterations. Total interpolation time was accumulated across iterations.

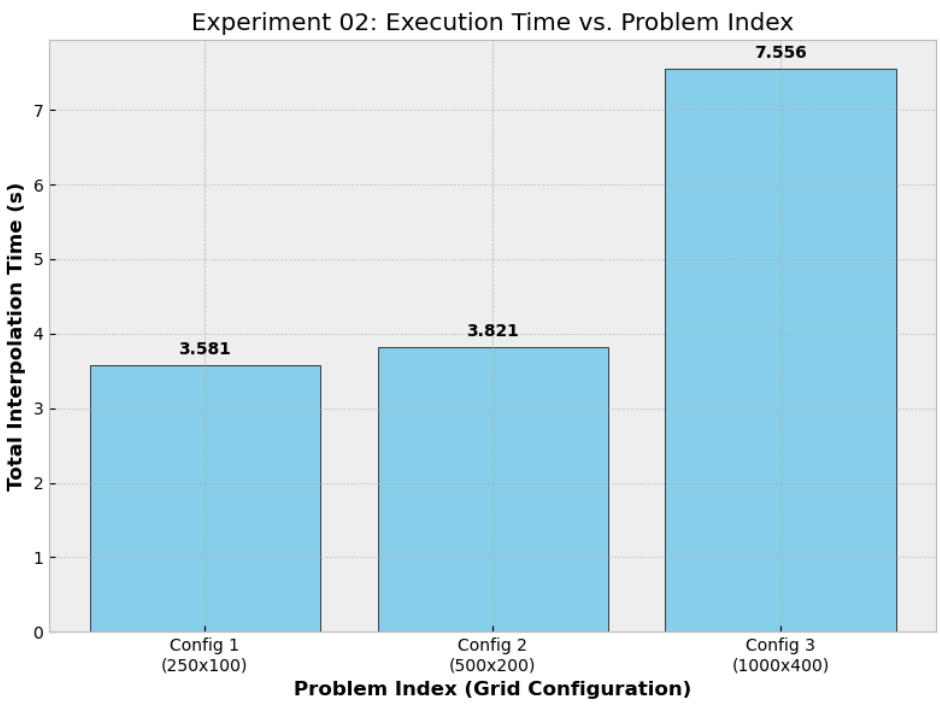

Figure 4: Interpolation consistency across the three grid configurations.

5 Experiment 03: The Mover Operation and OpenMP Parallelization

For a grid of $N_x = 1000$, $N_y = 400$, and 14 million particles, the Mover phase was introduced and timed individually. The Mover loop was parallelized using OpenMP with 4 threads. Speedup was computed as $S = T_{serial}/T_{parallel}$.

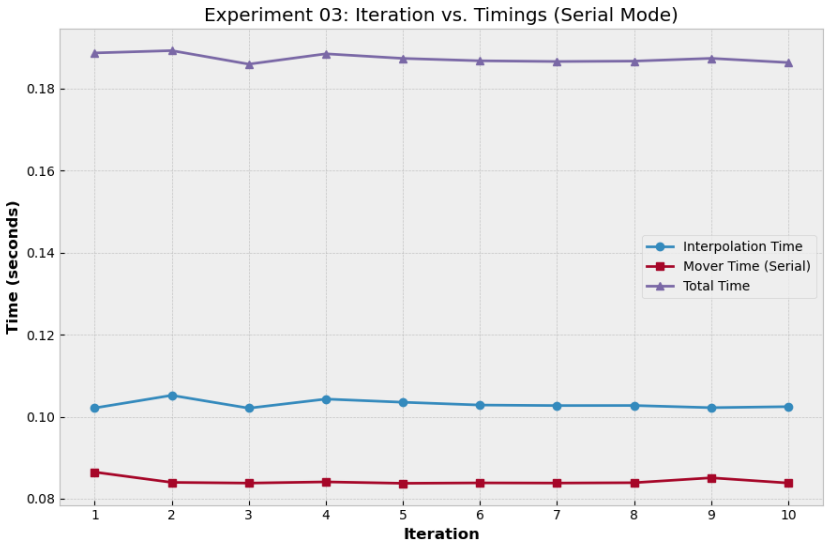

Figure 5: Iteration vs. Interpolation Time, Mover Time, and Total Time

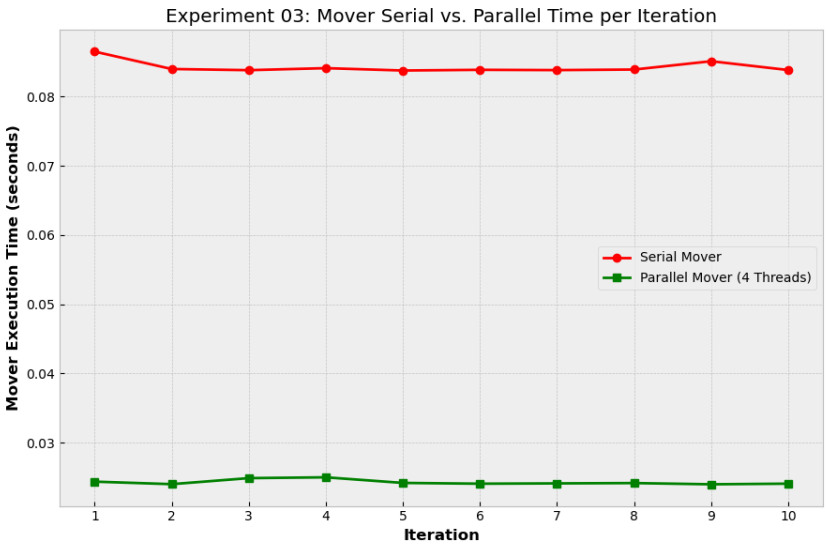

Figure 6: Serial vs. Parallel execution times for the Mover operation

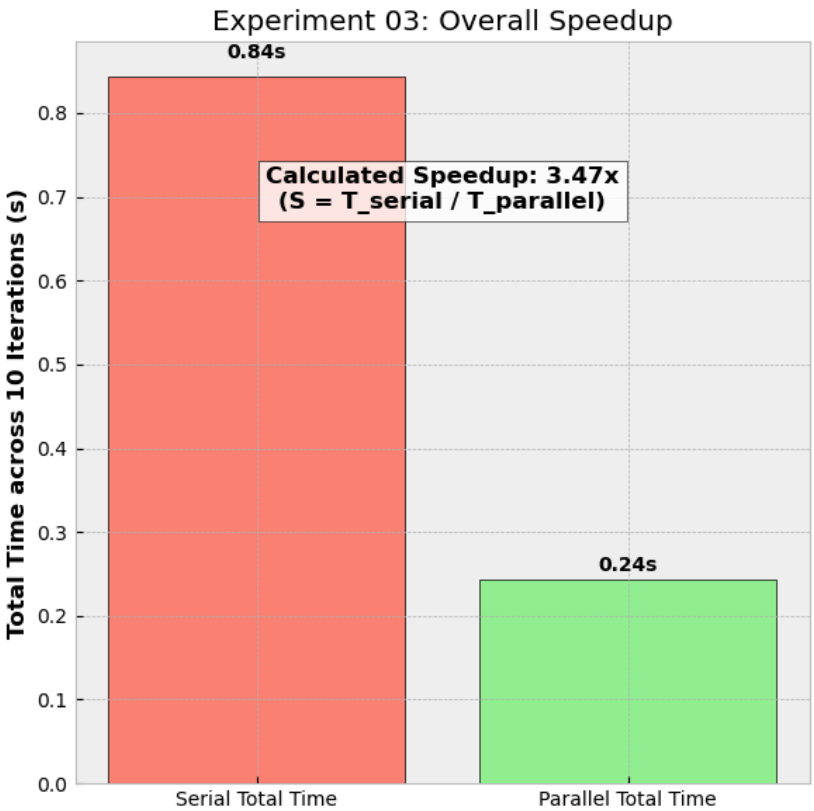

Figure 7: Speedup achieved by the OpenMP parallel Mover implementation

6 Analysis and Discussion

6.1 Architectural Performance Differences (Lab PC vs HPC Cluster)

Based on our observations, the execution times differ noticeably between the Lab PC and the HPC cluster due to their distinct architectural priorities:

- **Lab PC Performance:** The desktop PC typically utilizes a CPU optimized for high single-core frequency (e.g., 3.5+ GHz). This heavily benefits serial workloads like our interpolation phase, resulting in lower execution times.
- **HPC Cluster Performance:** The cluster nodes are designed for throughput, often featuring lower base clock speeds (e.g., 2.0-2.5 GHz) but massive multi-core capacity.
- **Conclusion:** Because the interpolation code in Experiments 01 and 02 is purely serial and heavily memory-bound, it is constrained by individual core speed and cache latency rather than parallel multi-core throughput, giving the Lab PC an advantage in this specific test.

6.2 OpenMP Mover Speedup and Hardware Observations

The Mover operation achieved a sub-linear speedup when parallelized. This is due to several hardware and implementation factors:

- **Speedup Calculation:** Speedup was calculated using the formula $S = \frac{T_{serial}}{T_{parallel}}$.
- **Memory Bandwidth Saturation:** Even though 4 threads were utilized, perfect 4x scaling is prevented by hardware limits. Every thread must continuously read from and write to the large particle array in memory, which quickly saturates the memory bus.
- **Overhead:** Maintaining thread-safe random number generation introduces minor operational overhead that impacts the maximum theoretical scaling.

6.3 Bottlenecks in Parallelizing Interpolation

If tasked with parallelizing the Interpolation function using `#pragma omp parallel for`, several critical bottlenecks would arise:

- **Race Conditions:** When multiple scattered points fall within the same cell or adjacent cells, multiple threads will attempt to update the shared grid points concurrently, leading to data corruption.
- **Atomic Operation Overhead:** To safely parallelize this, atomic operations (e.g., `#pragma omp atomic`) or explicit locks would be required for every mesh update to serialize memory access.
- **Efficiency Drop:** Given the massive number of particles and the frequency of mesh updates, these atomic directives would cause massive thread contention, drastically crippling the parallel performance and making the approach highly inefficient.

7 Conclusion

The integration of a Mover phase with OpenMP parallelization successfully demonstrates how computational bottlenecks can be isolated and optimized. However, the interpolation phase highlights the limits of multi-threading when shared memory architectures encounter heavy write-contention and race conditions.